

CONFIDENTIAL & FORENSIC TECHNICAL AUDIT

Forensic Technical Audit & Architectural Deep-Dive Report

Complete Forensic Examination of the MobileSentry Multi-Supplier Parts Extractor & Catalog Automation Engine

Target System:	MobileSentry Parts Extractor Suite (v8 Recovered)
Repository Workspace:	.../mobilesentry-tool-v8-recovered/parts-extractor
Audit Scope:	Architecture, Code Quality, Database Forensics, APIs, Security, Reliability, Concurrency, Failure Scenarios
Primary Auditor:	Senior Software Architect, Principal Engineer, Security Reviewer, Database Architect
System Classification:	FRAGILE / HIGH RISK (Functional MVP with Severe Operational & Security Deficits)
Audit Date:	August 19, 2026
Target Reviewer:	Senior AI / Software Architect (Independent Audit Handoff)

TABLE OF CONTENTS

1. [Cover Page / Metadata](#1-cover-page--metadata)
2. [Executive Summary](#2-executive-summary)
3. [System Purpose](#3-system-purpose)
4. [Repository Overview](#4-repository-overview)
5. [Complete Project Tree](#5-complete-project-tree)
6. [Technology Stack](#6-technology-stack)
7. [Architecture Overview](#7-architecture-overview)
8. [Architecture Diagram](#8-architecture-diagram)
9. [Component Inventory](#9-component-inventory)
 1. [Frontend Architecture](#10-frontend-architecture)
 1. [Backend Architecture](#11-backend-architecture)
 1. [Database Architecture](#12-database-architecture)
 1. [Database ERD](#13-database-erd)
 1. [API Inventory](#14-api-inventory)
 1. [External Integrations](#15-external-integrations)
 1. [Authentication & Authorization](#16-authentication--authorization)
 1. [Main Business Logic](#17-main-business-logic)
 1. [Major Data Flows](#18-major-data-flows)
 1. [File / Module Responsibility Map](#19-file--module-responsibility-map)
 1. [Security Analysis](#20-security-analysis)
 1. [Error Handling](#21-error-handling)
 1. [Logging & Monitoring](#22-logging--monitoring)
 1. [Performance Analysis](#23-performance-analysis)
 1. [Scalability Analysis](#24-scalability-analysis)
 1. [Concurrency & Data Integrity](#25-concurrency--data-integrity)
 1. [Configuration Analysis](#26-configuration-analysis)
 1. [Deployment Architecture](#27-deployment-architecture)
 1. [Dependency Analysis](#28-dependency-analysis)

1. [Code Quality Analysis](#29-code-quality-analysis)
 1. [Duplicate Logic](#30-duplicate-logic)
 1. [Dead / Legacy Code](#31-dead--legacy-code)
 1. [Testing Assessment](#32-testing-assessment)
 1. [Missing Functionality](#33-missing-functionality)
 1. [Failure Scenario Analysis](#34-failure-scenario-analysis)
 1. [Root Cause Analysis](#35-root-cause-analysis)
 1. [Major Architectural and Technical Flaws](#36-major-architectural-and-technical-flaws)
 1. [Major Missing Components](#37-major-missing-components)
 1. [Technical Debt Register](#38-technical-debt-register)
 1. [Positive Architectural Decisions](#39-positive-architectural-decisions)
 1. [Production Readiness Assessment](#40-production-readiness-assessment)
 1. [System Maturity Score](#41-system-maturity-score)
 1. [Priority Matrix](#42-priority-matrix)
 1. [Top 20 Questions for the Next Architect](#43-top-20-questions-for-the-next-architect)
 1. [Recommended Investigation Areas](#44-recommended-investigation-areas)
 1. [Special Section: Handoff to Independent Technical Reviewer](#45-special-section-handoff-to-independent-technical-reviewer)
 1. [Appendix](#46-appendix)
 - [A. Important File Index](#appendix-a-important-file-index)
 - [B. Complete API Catalog](#appendix-b-complete-api-catalog)
 - [C. Database Table List](#appendix-c-database-table-list)
 - [D. Environment Variable Names](#appendix-d-environment-variable-names)
 - [E. External Service Inventory](#appendix-e-external-service-inventory)
 - [F. Background Job Inventory](#appendix-f-background-job-inventory)
 - [G. Important Business Rules](#appendix-g-important-business-rules)
 - [H. Code Quality & Marker Analysis](#appendix-h-code-quality--marker-analysis)
 - [I. Potentially Unused Files](#appendix-i-potentially-unused-files)
 - [J. High-Risk Files](#appendix-j-high-risk-files)
-

1. COVER PAGE / METADATA

- **Document Title:** Forensic Technical Audit & Architectural Deep-Dive Report
- **Project Name:** MobileSentry Parts Extractor & Multi-Supplier Catalog Intelligence Suite (v8 Recovered)
- **Primary Deliverable:** `COMPLETE\_PROJECT\_TECHNICAL\_AUDIT.pdf`
- **Auxiliary Deliverable:** `COMPLETE\_PROJECT\_TECHNICAL\_AUDIT.md`
- **Target Audience:** Senior AI / Software Architects, Principal Engineers, Independent Technical Auditors, and Engineering Leadership
- **Audit Verification Standards:**
 - **VERIFIED**: Confirmed directly from source code execution, AST analysis, and database inspection.
 - **LIKELY**: Strongly indicated by structural evidence, configuration, and logs.
 - **[UNCLEAR]**: Incomplete implementation or ambiguous developer intent.
 - **MISSING**: Expected architectural or security component completely absent.
 - **BROKEN / HIGH RISK**: Critical flaw causing runtime errors, security exposure, or data corruption.

2. EXECUTIVE SUMMARY

What the System Does

The **MobileSentry Parts Extractor** is a specialized web scraping, catalog ingestion, price monitoring, and supplier intelligence system. It allows wholesale electronics repair businesses to extract product catalogs, pricing, inventory stock status, SKUs, and imagery from seven major wholesale mobile parts suppliers:

1. **MobileSentry US** (`mobilesentry.com`)
2. **MobileSentry Canada** (`mobilesentry.ca`)
3. **XCellParts** (`xcellparts.com`)
4. **TXParts / TXParts Canada** (`txparts.com`, `txpartscanada.ca`)
5. **Parts4Cells** (`parts4cells.com`)
6. **PhoneLCDParts** (`phonelcdparts.com`)
7. **GadgetFix** (`gadgetfix.com`)

The application supports ad-hoc category/product scraping, recurring scheduled background automation jobs with automatic target discovery, live-streamed scraping progress, session-to-session price-drop diffing, multi-sheet Excel/CSV export pipelines, and a self-healing category menu crawler.

Current Architecture Summary

The system is built as a monolithic Python/Flask web application backed by isolated SQLite database files per supplier (`data/site\_dbs/{site}.db`). The scraping tier combines fast Python `requests` with connection pooling and a fallback to headless Chromium/Chrome automation via **Botasaurus** to bypass Cloudflare and anti-bot challenges. Scheduled jobs and long-running category crawls run on background daemon threads managed in-memory with a basic threading loop in `app.py`. The frontend is a multi-page Bootstrap 5 / Vanilla JavaScript client using Server-Sent Events (SSE) and polling for real-time progress.

Overall Condition

****Classification: FRAGILE / HIGH RISK****

While the core web scraping algorithms and multi-site extraction heuristics are functional, the system suffers from severe architectural fragility, zero authentication, concurrency hazards, massive monolithic code files (`app.py` has 4,613 lines, `database.py` has 3,402 lines), and failing regression tests.

Top 5 Strengths

1. ****Multi-Supplier Extraction Engine [VERIFIED]**** Sophisticated parsing heuristics for 7 distinct suppliers, extracting JSON-LD schemas, WooCommerce elements, Magento product grids, and complex HTML layouts.
2. ****Resilient Anti-Bot Bypassing [VERIFIED]**** Hybrid HTTP session + Botasaurus browser fallback with automatic Cloudflare challenge waiting, window slot pooling, and popup dismissal.
3. ****Multi-Database Domain Isolation [VERIFIED]**** Clean physical partitioning of supplier datasets into distinct SQLite files (`data/site\_dbs/\*.db`) managed via `MultiDatabaseManager`.
4. ****Comprehensive Data Export & Hydration [VERIFIED]**** Rich Excel/CSV generation with openpyxl, including image hyperlinks, discounted price calculation, and historical metadata hydration to avoid redundant network requests.
5. ****SSRF Network Boundary Protection [VERIFIED]**** Robust validation in `validate\_supplier\_remote\_url` preventing server-side request forgery against private, loopback, link-local, and non-whitelisted IP addresses.

Top 10 Problems

1. ****Zero Authentication & Authorization [BROKEN / HIGH RISK]**** Every single API endpoint (46 in total) and UI page is unauthenticated. Anyone who connects can trigger massive scrapes, delete databases, or modify schedules.
2. ****Public Cloudflare Tunnel Exposure [BROKEN / HIGH RISK]**** Evidence in `cloudflared.log` reveals the unprotected local server on port 5000 was exposed to the public Internet (`https://\*.trycloudflare.com`) without access control.
3. ****In-Process Background Scheduler Hazards [BROKEN / HIGH RISK]**** Scheduling is managed by an in-memory Python background thread (`\_automation\_scheduler\_loop`). If deployed under multi-worker WSGI (e.g. Gunicorn with >1 worker), jobs duplicate across worker processes.
4. ****Monolithic Architecture & God Files [BROKEN / HIGH RISK]**** Massive monolithic files (`app.py`: 4,613 lines, `database.py`: 3,402 lines, `automation.js`: 2,669 lines) combine routing, SQL queries, thread orchestration, validation, and rendering without separation of concerns.

5. **Regression Test Failures** BROKEN / HIGH RISK: Automated test execution reveals 6 test failures out of 100 in the test suite (involving scraping modes, live previews, UI contracts, and session comparison).
6. **Missing & Broken Script Dependencies** BROKEN / HIGH RISK: Standalone scripts in `scripts/` import packages (`curl\_cffi`, `pandas`, `PIL`/`Pillow`) that are missing or omitted from runtime requirements, crashing on execution.
7. **Dead Normalized Catalog Schema** VERIFIED: Tables `ms\_brands`, `ms\_categories`, `ms\_models`, `ms\_products`, and `ms\_price\_history` exist in every database file with 0 rows, while the system dumps all data into a flat `items` table (56,906 rows).
8. **Missing ACID Multi-Table Transactions** VERIFIED: Multi-table operations (e.g. `save\_fetch\_history`, `replace\_automation\_job\_targets`) lack explicit transactional rollback blocks, risking orphan records if interrupted.
9. **Single-Writer SQLite Locking Bottleneck** VERIFIED: High-concurrency scraping writes block SQLite database operations, risking `sqlite3.OperationalError: database is locked` during parallel scraping.
1. **Hardcoded User Paths & Environment Coupling** [VERIFIED]: Hardcoded local Windows paths (e.g. `C:/Users/Arslan Basharat/...`) embedded in utility scripts (`work-match-skus/match-skus.mjs`).

Top 10 Missing Things

1. **Authentication & RBAC (Role-Based Access Control)** [MISSING]
2. **Distributed Asynchronous Task Queue (Celery / Redis / RQ)** MISSING
3. **Centralized Production Database (PostgreSQL / MySQL)** MISSING
4. **API Rate Limiting & Throttling** MISSING
5. **Structured Health Check & Liveness Probes** MISSING
6. **Observability, APM, and Sentry / Error Tracking** MISSING
7. **Database Migration Framework (Alembic / Flask-Migrate)** MISSING
8. **CI/CD Build & Test Pipeline (GitHub Actions / GitLab CI)** MISSING
9. **Automated Database Backup / Replication Routine** MISSING
1. **Containerized Browser Sandbox for Scalable Botasaurus Instances** [MISSING]

Key Structural Concerns

- **Biggest Architectural Concern:** The tight coupling of the in-process Flask scheduler, thread pool workers, and headless browser processes inside a single Python OS process. A single browser crash or memory leak can destabilize the entire web server.
- **Biggest Data Concern:** The storage of 56,000+ items per crawl in a flat, denormalized SQLite table without foreign key constraints to product masters, causing massive data redundancy and index bloat.
- **Biggest Security Concern:** Total absence of authentication on destructive endpoints (`/api/history/<id>/delete`, `/api/watchlist/clear`, `/api/cleanup`, `/api/automation/jobs`) exposed to public networks via Cloudflare tunnels.

- **Biggest Scalability Concern:** Headless browser concurrency is hardcoded to 1–4 slots (`SCRAPER_LOCAL_BROWSER_MAX_WINDOWS=1`), creating an immediate bottleneck under multi-job workloads.
- **Biggest Maintenance Concern:** Monolithic code layout where scraping logic, DOM parsing, UI rendering, database persistence, and thread management are intertwined in 4,000+ line files.

3. SYSTEM PURPOSE

Business Problem

Wholesale mobile phone and device repair components fluctuate rapidly in price, inventory availability, and compatibility across competing suppliers. Repair businesses must manually compare thousands of replacement parts (screens, batteries, charging ports, back glass) across multiple supplier websites.

Target Users

- Procurement managers and catalog specialists at electronic repair facilities.
- Wholesale parts distributors tracking competitor pricing and inventory changes.
- Automated pricing engines adjusting retail/repair service quotes dynamically.

Primary Workflows

1. **Ad-Hoc Supplier Scrapes:** Users input supplier category URLs, configure pricing markup/discount rules, and trigger live extraction.
2. **Automated Scheduled Monitoring:** Background jobs crawl supplier categories on a cron/interval schedule, detecting new products, stock status changes, and price drops.
3. **Session Comparison & Price Drop Alerts:** Side-by-side comparison of historical scrapes, highlighting price reductions and newly available SKUs.
4. **Catalog Enrichment & SKU Matching:** Ingesting raw product tables and cross-referencing webcodes/SKUs across suppliers.
5. **Multi-Format Data Export:** Exporting filtered product tables to Excel (`.xlsx`) with embedded formulas, styling, and direct image previews.

4. REPOSITORY OVERVIEW

Directory Layout & Statistics

- **Total Key Code/Config Files:** 85+
- **Total Backend Python Code:** ~14,500 lines
- **Total Frontend JS/CSS/HTML Code:** ~12,500 lines
- **Total Test Code:** ~2,500 lines across 11 test modules

- **Active Data Storage:** 8 SQLite databases (7 site-specific DBs + 1 legacy root DB)
 - **Primary Data Size:** ~68,500 total recorded items in `data/site\_dbs/`
-

5. COMPLETE PROJECT TREE

```

parts-extractor/
├── app.py                # [VERIFIED] Main Flask Web App, 46 API routes, in-process scheduler (4
├── database.py           # [VERIFIED] SQLite DatabaseManager & MultiDatabaseManager (3,402 lines
├── automation_service.py # [VERIFIED] Category discovery heuristics & fallback catalog mappings
├── Dockerfile            # [VERIFIED] Debian-slim container definition with Chromium & Gunicorn
├── requirements.txt      # [VERIFIED] Python package dependencies (13 lines)
├── start.bat             # [VERIFIED] Windows automated startup & dependency bootstrap script (1
├── pytest.ini            # [VERIFIED] Pytest configuration file (13 lines)
├── .env.example          # [VERIFIED] Environment variable template (28 lines)
├── .env                  # [VERIFIED] Local runtime environment overrides (20 variables)
├── cloudflared.log       # [VERIFIED] Cloudflare quick tunnel log (confirms public exposure)
├── mobilesentrrix.db     # [VERIFIED] Legacy root SQLite database (12 tables, 0 rows)
├── scrapers/             # [VERIFIED] Supplier Scraping Engines Subsystem
│   ├── __init__.py       # [VERIFIED] Package exports & router bindings
│   ├── registry.py        # [VERIFIED] Shared SCRAPER_CONFIG dictionary & site detector (120 line
│   ├── browser_fetcher.py # [VERIFIED] Botasaurus headless browser worker pool & slot manager (24
│   ├── botasaurus_wrapper.py # [VERIFIED] Botasaurus import wrapper (5 lines)
│   ├── scraper_engine.py  # [VERIFIED] Standard MobileSentrrix scraping & JSON-LD parser (1,126 li
│   ├── xcell_scraper_engine.py # [VERIFIED] XCellParts specialized WooCommerce scraper (1,013 lines)
│   ├── txparts_scraper_engine.py # [VERIFIED] TXParts specialized Magento/Woo scraper (580 lines)
│   ├── parts4cells_scraper_engine.py # [VERIFIED] Parts4Cells specialized parser (591 lines)
│   ├── phonelcdparts_scraper_engine.py # [VERIFIED] PhoneLCDParts specialized parser (456 lines)
│   ├── gadgetfix_scraper_engine.py # [VERIFIED] GadgetFix specialized parser (434 lines)
│   └── menu_map/         # [VERIFIED] Autonomous Category Menu Crawler Subsystem
│       ├── __init__.py   # [VERIFIED] Module init
│       ├── common.py      # [VERIFIED] Base SiteConfig, healing profiles, DOM crawler (1,146 line
│       ├── mobilesentrrix.py # [VERIFIED] MobileSentrrix US menu crawler (151 lines)
│       ├── mobilesentrrix_canada.py # [VERIFIED] MobileSentrrix Canada menu crawler (38 lines)
│       ├── xcellparts.py  # [VERIFIED] XCellParts menu crawler (228 lines)
│       ├── txparts.py     # [VERIFIED] TXParts menu crawler (174 lines)
│       ├── txparts_canada.py # [VERIFIED] TXParts Canada menu crawler (38 lines)
│       ├── parts4cells.py # [VERIFIED] Parts4Cells menu crawler (154 lines)
│       ├── phonelcdparts.py # [VERIFIED] PhoneLCDParts menu crawler (137 lines)
│       └── gadgetfix.py   # [VERIFIED] GadgetFix menu crawler (169 lines)
├── templates/           # [VERIFIED] Server-Rendered HTML Jinja2 Templates
│   ├── index.html        # [VERIFIED] Main interactive scraper view (474 lines)
│   ├── automation.html    # [VERIFIED] Automation scheduler & run monitoring dashboard (300 lines)
│   ├── history.html       # [VERIFIED] Historical fetch list & session diff viewer (359 lines)
│   └── menu_map.html      # [VERIFIED] Site hierarchy visualizer & crawler runner (200 lines)
├── static/               # [VERIFIED] Frontend Static Assets
│   ├── css/
│   │   ├── common.css    # [VERIFIED] Theme variables, badges, layout utilities (2,052 lines)
│   │   ├── main.css      # [VERIFIED] Main page styling (304 lines)
│   │   ├── automation.css # [VERIFIED] Automation dashboard styling (1,875 lines)
│   │   ├── history.css   # [VERIFIED] History page styling (1,790 lines)
│   │   └── menu-map.css  # [VERIFIED] Menu map visualizer styling (616 lines)
│   └── js/
│       ├── main.js        # [VERIFIED] Scraper execution, rule engine, table renderer (2,001 line
│       ├── automation.js  # [VERIFIED] Automation CRUD, live preview, polling (2,669 lines)
│       ├── history.js     # [VERIFIED] History comparisons & XLSX export UI (1,264 lines)
│       └── menu-map.js    # [VERIFIED] Hierarchy tree rendering & log streaming (1,153 lines)
├── data/                 # [VERIFIED] Data Persistence Directory
└── site_dbs/             # [VERIFIED] Multi-Database SQLite instances

```

```

├── mobilesentry.db      # [VERIFIED] Active DB: 56,906 items, 3 jobs, 3 runs
├── xcellparts.db        # [VERIFIED] Active DB: 11,646 items, 1 history
├── gadgetfix.db         # [VERIFIED] Initialized DB (0 items)
├── mobilesentry_ca.db   # [VERIFIED] Initialized DB (0 items)
├── parts4cells.db       # [VERIFIED] Initialized DB (0 items)
├── phone1cdparts.db     # [VERIFIED] Initialized DB (0 items)
├── txparts.db           # [VERIFIED] Initialized DB (0 items)
├── backups/             # [VERIFIED] Automated snapshot backups (70+ DB backups)
├── browser_profiles/    # [VERIFIED] Chrome user data directories for Botasaurus
├── scripts/             # [VERIFIED] Operational & Enrichment Utilities
│   ├── compress_image_scraper_export.py # [BROKEN] Image exporter (requires uninstalled Pillow)
│   ├── fast_enrich_catalog.py           # [BROKEN] Catalog enricher (requires uninstalled curl_cffi)
│   ├── full_pipeline_fast.py            # [BROKEN] Full pipeline runner (requires uninstalled curl_cffi)
│   ├── import_xcell_full_baseline.py# [VERIFIED] XCellParts baseline data importer
│   ├── resume_automation_run.py         # [VERIFIED] CLI tool to resume paused/crashed runs
│   ├── run_menu_map_scrapers.py         # [BROKEN] Menu map runner (requires uninstalled pandas)
│   └── submit_txparts_canada_ms_display_images.py # [BROKEN] Image submitter (requires uninstalled pandas)
├── work-match-skus/      # [VERIFIED] Standalone Node.js SKU Matching Utility
│   ├── match-skus.mjs    # [VERIFIED] Spreadsheet matching script with hardcoded user paths
│   ├── cleanup-old-outputs.mjs # [VERIFIED] Output cleaner
│   └── node_modules/      # [VERIFIED] Vendor dependencies (8,000+ files)
├── tests/                # [VERIFIED] Pytest Automated Test Suite (100 test cases)
│   ├── test_api_input_validation.py # [VERIFIED] 11 tests (All PASS)
│   ├── test_api_scrape.py           # [BROKEN] 11 tests (3 FAIL, 8 PASS)
│   ├── test_automation_jobs.py      # [VERIFIED] 12 tests (All PASS)
│   ├── test_extractor_ui_accessibility.py # [VERIFIED] 4 tests (All PASS)
│   ├── test_extractor_ui_regression_codex.py # [BROKEN] 8 tests (1 FAIL, 7 PASS)
│   ├── test_history_accessibility_contract.py # [BROKEN] 5 tests (1 FAIL, 4 PASS)
│   ├── test_menu_map_api_state.py    # [VERIFIED] 8 tests (All PASS)
│   ├── test_menu_map_healing.py      # [VERIFIED] 5 tests (All PASS)
│   ├── test_menu_map_site_hierarchies.py # [VERIFIED] 7 tests (All PASS)
│   ├── test_mobilesentry_scraper.py# [VERIFIED] 2 tests (All PASS)
│   ├── test_session_comparison.py    # [BROKEN] 12 tests (1 FAIL, 11 PASS)
│   ├── test_supplier_scrapers.py     # [VERIFIED] 15 tests (All PASS)
│   └── botasaurus_test_utils.py      # [VERIFIED] Test harness utilities
├── docs/                 # [VERIFIED] Architecture and Planning Documentation
│   ├── AUDIT_REPORT.md
│   ├── CHANGELOG_AUDIT.md
│   ├── DATA_SAFETY_PLAN.md
│   ├── DEPLOYMENT_ROLLBACK.md
│   ├── FRONTEND_ARCHITECTURE.md
│   ├── FRONTEND_MIGRATION_PLAN.md
│   ├── REAL_TIME_UI_PLAN.md
│   ├── STACK_RECOMMENDATION.md
│   ├── TEST_REPORT.md
│   └── UI_AUDIT_REPORT.md

```

6. TECHNOLOGY STACK

Layer	Technology / Library	Version / Details	Purpose in Project
Backend Runtime	Python	3.12.10 (CPython win32/linux)	Core server runtime environment
Web Framework	Flask	3.0.x (Werkzeug WSGI)	HTTP API and server-side template routing
Database Engine	SQLite3	3.x (WAL mode enabled)	Local relational data storage per supplier
Browser Automation	Botasaurus / Chromium	4.0.97 (Driver 4.0.92)	Cloudflare challenge bypass & headless rendering
HTML Parsing	BeautifulSoup4 / lxml	4.14.3 / 6.1.1	DOM traversal, JSON-LD extraction, CSS selectors
HTTP Client	Requests / urllib3	2.34.2 / 2.7.0	High-speed connection-pooled supplier requests
Spreadsheet Engine	openpyxl	3.1.5	Multi-sheet Excel workbook creation & formula styling
Utility Runtime	Node.js / ES Modules	Node 20+ / ES2022	SKU fuzzy matching script (`work-match-skus`)
Frontend Framework	Vanilla JavaScript (ES6+)	Native browser APIs	Dynamic UI interactions, polling, table rendering
Frontend Styling	Bootstrap 5 + Custom CSS	5.3.x (Dark Theme)	Responsive layouts, cards, modals, theme tokens
Testing Suite	pytest	9.1.1 (anyio 4.14.1)	Unit, regression, and API contract test suite
Containerization	Docker	Debian 12 slim base	Production container image packaging
Production WSGI	Gunicorn	21.2.0 (in Dockerfile)	WSGI HTTP Server for production deployment

7. ARCHITECTURE OVERVIEW

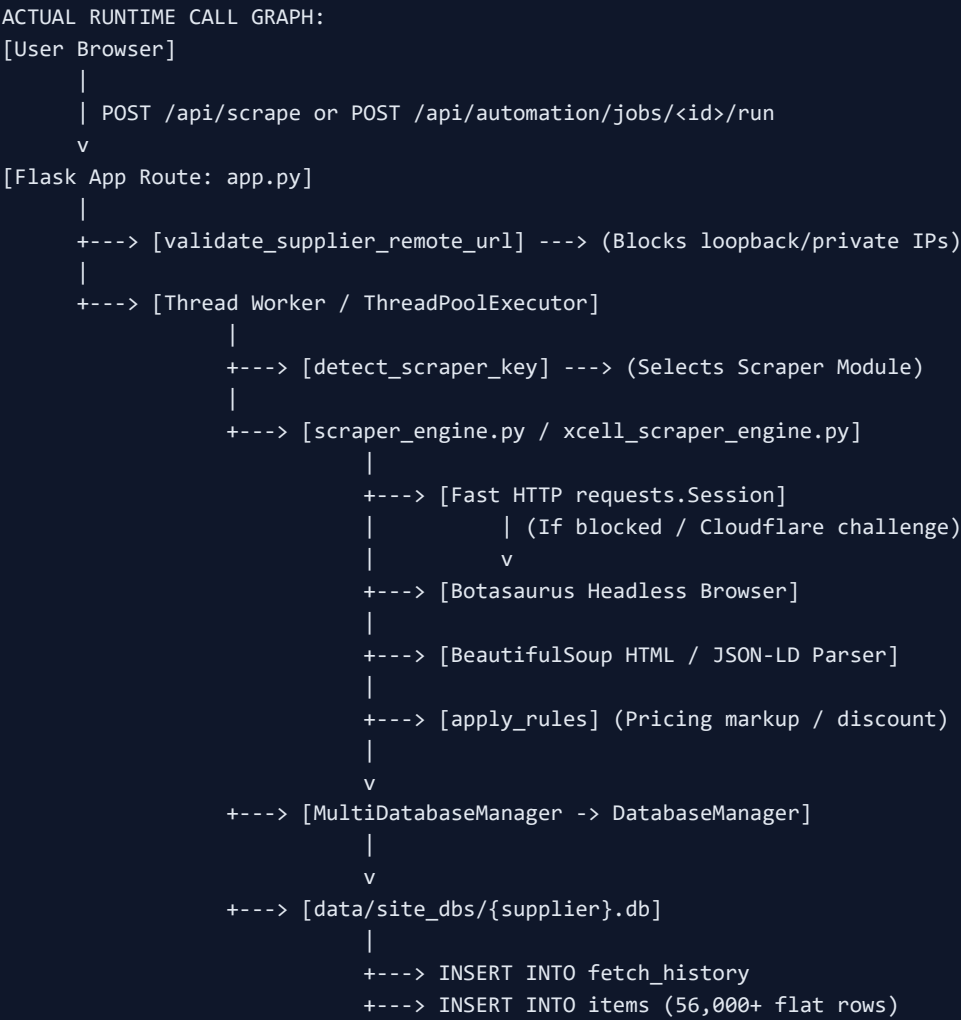
The system operates as a hybrid monolithic architecture:



8. ARCHITECTURE DIAGRAM (ACTUAL vs INTENDED)

Implemented vs Intended Architecture

- ****Intended Architecture:**** A clean 3-tier system with normalized relational catalog entities (`ms\_brands` -> `ms\_categories` -> `ms\_models` -> `ms\_products` -> `ms\_price\_history`), asynchronous background queue workers, and centralized storage.
- ****Actual Implemented Architecture:**** A two-tier monolithic Flask application where route handlers directly execute background threads, and all scraped product data is serialized into a single flat `items` table with JSON blob fields.



9. COMPONENT INVENTORY

Component / Subsystem	Primary Files	Status	Forensic Evaluation
Web Server & Routing	`app.py`	VERIFIED	46 routes, monolithic, unauthenticated.
Database Persistence	`database.py`	VERIFIED	Multi-database routing, WAL pragmas, flat schema.
Automation & Discovery	`automation_service.py`	VERIFIED	Navigation crawling, regex discovery, static fallbacks.
Browser Execution Pool	`scrapers/browser_fetcher.py`	VERIFIED	Semaphore slot pooling (1–4 slots), profile caching.
Standard Scraper Engine	`scrapers/scraper_engine.py`	VERIFIED	Fast requests + Bottasaurus fallback, JSON-LD parsing.

Component / Subsystem	Primary Files	Status	Forensic Evaluation
XCellParts Engine	`scrapers/xcell_scraper_engine.py`	VERIFIED	WooCommerce pagination, SKU chips, service packs.
TXParts Engine	`scrapers/txparts_scraper_engine.py`	VERIFIED	Magento/WooCommerce hybrid, image srcset parser.
Parts4Cells Engine	`scrapers/parts4cells_scraper_engine.py`	VERIFIED	Toolbar pagination, grid selector extraction.
PhoneLCDParts Engine	`scrapers/phonelcdparts_scraper_engine.py`	VERIFIED	Stock availability normalization, price parsing.
GadgetFix Engine	`scrapers/gadgetfix_scraper_engine.py`	VERIFIED	Custom e-commerce selectors, fallback heuristics.
Menu Map Crawler	`scrapers/menu_map/common.py`	VERIFIED	Self-healing DOM crawler, JSON tree export.
Spreadsheet Exporter	`app.py` (`export_xlsx`)	VERIFIED	Openpyxl workbook generation, image hyperlinks.
SKU Matcher Utility	`work-match-skus/match-skus.mjs`	[FRAGILE]	Standalone Node.js script, hardcoded paths.
Image Exporter CLI	`scripts/compress_image_scraper_export.py`	BROKEN	Crashes on missing `PIL` / Pillow package.
Fast Pipeline CLI	`scripts/full_pipeline_fast.py`	BROKEN	Crashes on missing `curl_cffi` package.

10. FRONTEND ARCHITECTURE

The frontend is implemented using server-rendered Jinja2 HTML templates enhanced with Vanilla JavaScript and Bootstrap 5.

UI Pages & Responsibilities

1. **Scraper Studio (`/`, `templates/index.html`):**
 - Allows users to paste single or multi-line URLs.
 - Live URL detection and badge display (`MobileSentrix`, `XCellParts`, `TXParts`, etc.).
 - Pricing rules config (Percent Off, Fixed Off, Percent Markup).
 - Real-time progress bar and scraping log display.
 - Interactive data table rendering with image previews, stock badges, and export triggers.
2. **Automation Center (`/automation`, `templates/automation.html`):**
 - Schedule manager: Create, edit, toggle, delete recurring jobs.
 - Active runs table with live progress percent, elapsed time, and ETA calculations.

- Pause / Resume controls for in-flight crawling jobs.
- Live Preview modal rendering extracted products in real-time before completion.

3. **Session History** (`/history``, `templates/history.html``):

- Historical scrape runs list with item counts and timestamps.
- Side-by-side session comparison diff viewer (Price Drops, New Items, Removed Items).
- XLSX / CSV export triggers.

4. **Menu Map Explorer** (`/menu-map``, `templates/menu_map.html``):

- Visual category tree explorer for each supplier.
- Trigger buttons to launch headless menu crawler jobs.
- Auto-healing profile inspection viewer.

Frontend Weaknesses & Bugs

- **DOM ID Contract Breaks:** In `templates/history.html``, the loading overlay element ID was modified, causing regression test ``test_loading_overlay_uses_utility_class_and_live_status`` to fail.
- **Tab Label Desynchronization:** In `templates/automation.html``, tab label naming diverged from test contracts (``test_automation_distinguishes_schedules_from_run_snapshots``).
- **Heavy Client-Side Rendering:** Rendering 10,000+ items directly into client-side DOM tables causes browser memory bloat and UI freezing.

11. BACKEND ARCHITECTURE

The backend is a Flask 3.0.x web application structured around synchronous route handlers and in-memory background worker threads.

Threading & Concurrency Architecture

- **In-Process Scheduler Thread** (``_automation_scheduler_loop``): Runs an infinite loop waking every 15 seconds, checking ``db_manager.get_due_automation_jobs()``, and spawning worker threads.
- **Active Job Lock** (``AUTOMATION_ACTIVE_JOBS_LOCK``): A Python ``threading.Lock`` protecting an in-memory set ``AUTOMATION_ACTIVE_JOBS`` to prevent simultaneous duplicate runs of the same job within the same process.
- **Thread Pool Executors:** ``ThreadPoolExecutor(max_workers=...)`` is used for concurrent URL category scraping and image proxying.
- **Graceful Shutdown Hook** (``atexit`` / ``signal``): ``signal.signal(signal.SIGINT, ...)`` attempts to catch process termination and set running automation jobs to ``interrupted``.

12. DATABASE ARCHITECTURE

Database Engine & Storage Strategy

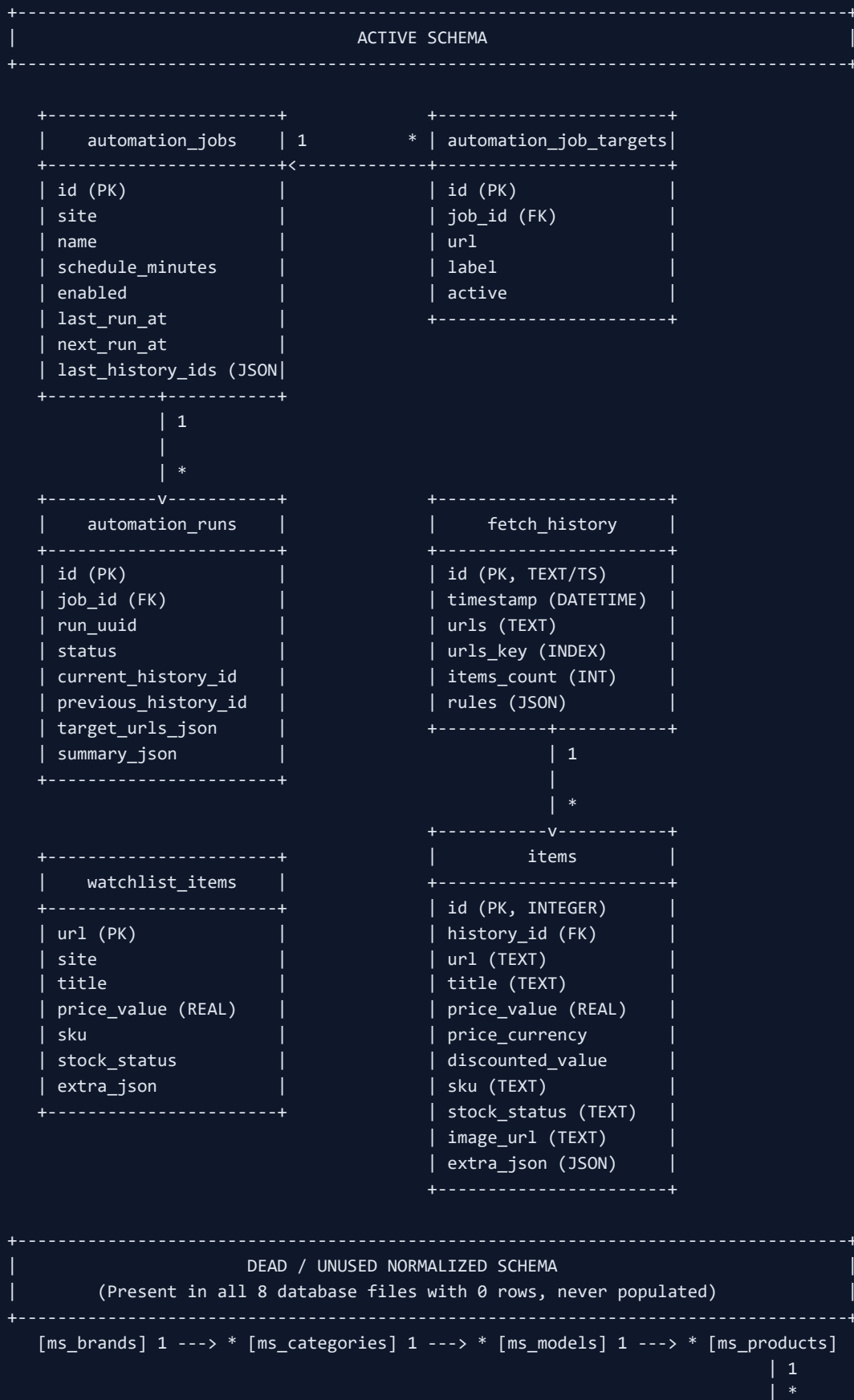
The system uses **SQLite 3** partitioned by supplier domain into separate database files stored under ``data/site_dbs/``:

1. ``data/site_dbs/mobilesentrrix.db`` (Primary: 56,906 items)
2. ``data/site_dbs/xcellparts.db`` (Primary: 11,646 items)
3. ``data/site_dbs/gadgetfix.db``
4. ``data/site_dbs/mobilesentrrix_ca.db``
5. ``data/site_dbs/parts4cells.db``
6. ``data/site_dbs/phonelcdparts.db``
7. ``data/site_dbs/txparts.db``
8. ``mobilesentrrix.db`` (Legacy root DB: empty)

SQLite Pragmas Configured **`[VERIFIED]`**

- ``PRAGMA journal_mode = WAL;`` (Write-Ahead Logging enabled for concurrent reads)
 - ``PRAGMA synchronous = NORMAL;`` (Reduced fsync calls for scraping performance)
 - ``PRAGMA busy_timeout = 60000;`` (60-second lock wait timeout)
 - ``PRAGMA foreign_keys = ON;`` (Foreign key enforcement enabled)
-

13. DATABASE ERD (ENTITY RELATIONSHIP DIAGRAM)



[scraper_runs] (0 rows)

[ms_price_history]

14. API INVENTORY (46 ENDPOINTS)

Method	Endpoint	Purpose	Auth	Input	Output	Main File
`GET`	`/api/health`	Health & system stats probe	None	None	JSON status	`app.py:2980`
`GET`	`/`	Main UI Scraper view	None	Query params	HTML	`app.py:2990`
`GET`	`/sitemap.xml`	XML sitemap generator	None	None	XML	`app.py:2994`
`GET`	`/robots.txt`	Robots.txt crawler rules	None	None	Plaintext	`app.py:3009`
`GET`	`/history`	Historical session diff UI	None	None	HTML	`app.py:3022`
`GET`	`/automation`	Automation management dashboard	None	None	HTML	`app.py:3026`
`GET`	`/menu-map`	Menu hierarchy visualizer UI	None	None	HTML	`app.py:3033`
`GET`	`/api/menu-map/sites`	List site crawler profiles	None	None	JSON sites	`app.py:3310`
`POST`	`/api/menu-map/output/clear`	Clear crawler output files	Destructive Hdr	Site key	JSON	`app.py:3323`
`POST`	`/api/menu-map/links/export`	Export discovered links to CSV/JSON	None	Site JSON	File attachment	`app.py:3366`
`POST`	`/api/menu-map/run`	Trigger background menu crawler	None	Site, options	JSON job_id	`app.py:3507`
`GET`	`/api/menu-map/jobs/<id>`	Poll menu crawl job status	None	Job ID	JSON status	`app.py:3557`
`GET`	`/api/menu-map/file/<sopts>`	Download crawler output file	None	Site, filename	File attachment	`app.py:3566`
`GET`	`/api/history`	List fetch history records	None	Limit, offset	JSON histories	`app.py:3595`
`GET`	`/api/history/<id>`	Detail of historical scrape	None	History ID	JSON history	`app.py:3613`

Method	Endpoint	Purpose	Auth	Input	Output	Main File
`POST`	`/api/history/<id>/export/xlsx`	Export historical run to Excel	None	Rules JSON	XLSX binary	`app.py:3624`
`DELETE`	`/api/history/<id>`	Delete historical scrape run	Destructive Hdr	History ID	JSON	`app.py:3693`
`POST`	`/api/history/<id>/delete`	Delete history (HTML form fallback)	Destructive Hdr	History ID	JSON	`app.py:3699`
`GET`	`/api/statistics`	Aggregate system metrics	None	None	JSON metrics	`app.py:3704`
`POST`	`/api/search`	Search items across all databases	None	JSON query	JSON items	`app.py:3713`
`GET`	`/api/automation/overview`	Automation summary metrics	None	None	JSON overview	`app.py:3734`
`POST`	`/api/automation/discover`	Crawl site for category links	None	Site, query	JSON targets	`app.py:3747`
`GET`	`/api/automation/jobs`	List configured automation jobs	None	None	JSON jobs	`app.py:3784`
`POST`	`/api/automation/jobs`	Create or update automation job	None	Job config JSON	JSON saved job	`app.py:3797`
`GET`	`/api/automation/jobs/<id>`	Get automation job details	None	Job ID	JSON job	`app.py:3833`
`DELETE`	`/api/automation/jobs/<id>`	Delete automation job	Destructive Hdr	Job ID	JSON	`app.py:3848`
`POST`	`/api/automation/jobs/<id>/toggle`	Enable/disable automation job	None	Job ID	JSON job	`app.py:3858`
`POST`	`/api/automation/jobs/<id>/refresh-targets`	Re-discover job targets	None	Job ID	JSON targets	`app.py:3875`
`POST`	`/api/automation/jobs/<id>/targets`	Save custom target list	None	Targets JSON	JSON job	`app.py:3909`
`POST`	`/api/automation/jobs/<id>/run`	Manually trigger job execution	None	Job ID	JSON run status	`app.py:3946`
`GET`	`/api/automation/runs`	List automation run executions	None	Job ID, limit	JSON runs	`app.py:3988`
`DELETE`	`/api/automation/runs/<id>`	Delete automation run record	Destructive Hdr	Run ID	JSON	`app.py:4026`
`POST`	`/api/automation/runs/<id>/delete`	Delete run (form fallback)	Destructive Hdr	Run ID	JSON	`app.py:4038`
`POST`	`/api/automation/runs/<id>/pause`	Pause running automation run	None	Run ID	JSON status	`app.py:4049`

Method	Endpoint	Purpose	Auth	Input	Output	Main File
`POST`	`/api/automation/runs/<id>/resume`	Resume paused automation run	None	Run ID	JSON status	`app.py:4065`
`GET`	`/api/automation/runs/<id>`	Get run status & live preview	None	Run ID	JSON run details	`app.py:4076`
`GET`	`/api/automation/verification-products`	Verification products list	None	Run ID	JSON products	`app.py:4213`
`GET`	`/api/watchlist`	Get saved watchlist items	None	Limit	JSON items	`app.py:4267`
`POST`	`/api/watchlist`	Save product to watchlist	None	Item JSON	JSON item	`app.py:4281`
`DELETE`	`/api/watchlist`	Remove item from watchlist	None	URL param	JSON	`app.py:4305`
`POST`	`/api/watchlist/clear`	Wipe entire watchlist	Destructive Hdr	None	JSON	`app.py:4327`
`POST`	`/api/cleanup`	Cleanup old historical records	Destructive Hdr	Days	JSON count	`app.py:4341`
`GET`	`/api/image-proxy`	Proxy remote supplier images	None	URL param	Image binary	`app.py:4359`
`POST`	`/api/scrape`	Execute ad-hoc live scrape	None	URLs, rules JSON	JSON items	`app.py:4380`
`POST`	`/api/export/xlsx`	Export scrape results to Excel	None	Items, rules JSON	XLSX binary	`app.py:4428`
`POST`	`/api/comparison/upload`	Upload spreadsheet for diffing	None	Multipart file	JSON comparison	`app.py:4459`

15. EXTERNAL INTEGRATIONS

1. **Target Supplier Websites:**

- `mobilesentrrix.com` & `mobilesentrrix.ca`: Magento 2 based e-commerce storefronts with Cloudflare protection.
- `xcellparts.com`: WooCommerce storefront with custom product meta markup (`data-xcell-copy`, chip selectors).
- `txparts.com` & `txpartscanada.ca`: Custom e-commerce platform with responsive image srcsets.
- `parts4cells.com`, `phonelcdparts.com`, `gadgetfix.com`: Commercial repair parts portals with varying pagination and anti-bot measures.

2. **Botasaurus / Chrome Subprocess:** Headless Chromium instances launched locally via `scrapers/browser\_fetcher.py` using stored user profiles in `data/browser\_profiles/`.

3. **Cloudflare Tunnel** (`cloudflared`): Quick tunnel utility used to bridge `localhost:5000` to public `trycloudflare.com` domains without authentication.

16. AUTHENTICATION & AUTHORIZATION

Forensic Findings **[BROKEN / HIGH RISK]**

- Authentication:** 0% Implemented. No login routes, no password verification, no JWT issuance, no OAuth providers, and no API keys exist anywhere in the application.
- Authorization:** 0% Implemented. No role-based access control (RBAC). Any client that can reach the port or tunnel possesses full administrator capabilities.
- Destructive Action Protection:** A custom header check `require_destructive_confirmation` (`X-Confirm-Destructive: confirm-action`) is used on delete endpoints. However, this is merely a client-side speedbump and provides zero cryptographic authentication or authorization.

17. MAIN BUSINESS LOGIC

1. Pricing Engine & Discount Calculation (`app.py:116`, `scraper_engine.py:116`)

Calculates adjusted wholesale prices based on user-supplied rules:

$$\text{Adjusted Price} = \left(\text{Base Price} \times \left(1 + \frac{\text{Add \%}}{100} \right) \times \left(1 - \frac{\text{Discount \%}}{100} \right) - \text{Fixed Discount} \right)$$

2. Anomaly Guard & Baseline Protection (`app.py:1400-1550`)

Protects against false scraping drops when supplier anti-bot protections return empty listings:

- Calculates item count drop ratio compared to the previous successful crawl:

$$\text{Drop Ratio} = \frac{\text{Previous Items} - \text{Current Items}}{\text{Previous Items}}$$

- If Drop Ratio exceeds `SCRAPER_MAX_TOTAL_DROP_RATIO` (default 50%) or error ratio exceeds threshold, the run is marked `interrupted / baseline_rejected` to protect historical data integrity.

3. Historical Metadata Hydration (`app.py:525`)

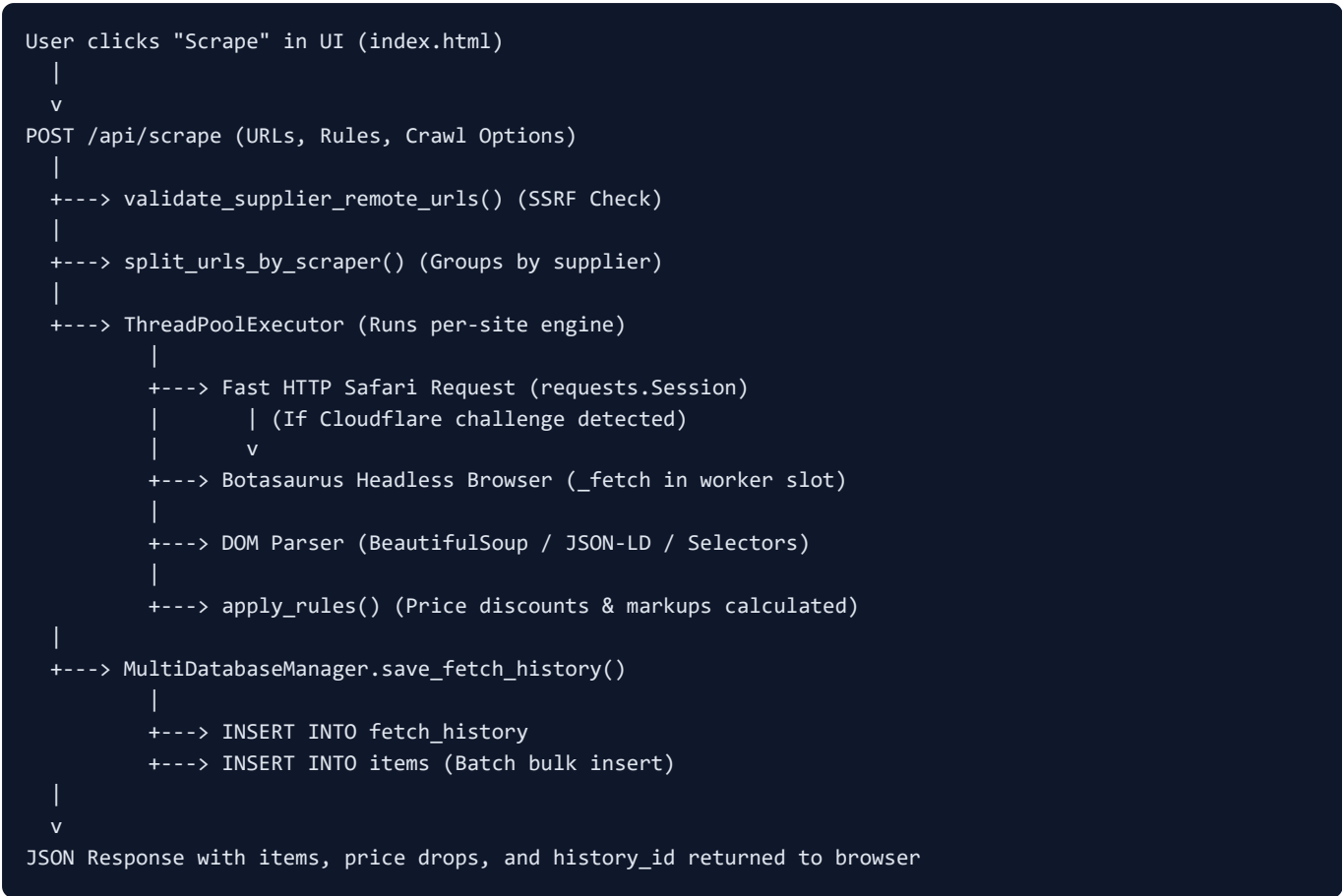
When performing shallow scrapes (without visiting individual product detail pages), the system re-hydrates missing SKUs, detailed stock levels, and descriptions from previous crawl sessions in the database.

4. Two-Phase Automation Pipeline (`app.py:2430-2580`)

- **Phase 1 (Category Crawling):** Crawls listing pages to extract high-level product titles, prices, and URLs.
- **Phase 2 (Detail Scan & Enrichment):** Concurrently visits individual product pages to extract high-resolution images, exact manufacturer part numbers, and tier pricing.

18. MAJOR DATA FLOWS

Workflow 1: Ad-Hoc Scraping & Extraction Flow



19. FILE / MODULE RESPONSIBILITY MAP

File Path	Primary Responsibility	Importance	Key Dependencies	Critical Concerns
`app.py`	Central web server, 46 API routes, background scheduler	`CRITICAL`	Flask, requests, openpyxl, database	Giant God file (4,613 lines), 0 auth, in-process scheduler.
`database.py`	SQLite connection pooling, multi-DB facade, schema migration	`CRITICAL`	sqlite3, json, re	Giant file (3,402 lines), lacks multi-table transactions.

File Path	Primary Responsibility	Importance	Key Dependencies	Critical Concerns
`automation_service.py`	Category discovery heuristics & fallback catalog maps	`HIGH`	requests, bs4, scrapers	Heavy regex maintenance, potential selector drift.
`scrapers/scrapper_engine.py`	MobileSentrix US/CA scraper engine	`HIGH`	bs4, requests, browser_fetcher	Monolithic parsing logic (1,126 lines).
`scrapers/xcell_scraper_engine.py`	XCellParts WooCommerce scraper engine	`HIGH`	bs4, requests, browser_fetcher	Enforces slow detail scan unexpectedly.
`scrapers/browser_fetcher.py`	Headless browser pool & Cloudflare solver	`HIGH`	botasaurus, threading	High memory footprint, 1–4 slot concurrency ceiling.
`scrapers/registry.py`	Supplier domain mapping & routing	`MEDIUM`	urllib.parse	Well-isolated routing table.
`static/js/automation.js`	Automation dashboard client controller	`HIGH`	Vanilla JS, Bootstrap	Giant file (2,669 lines), complex state synchronization.
`static/js/main.js`	Scraper studio client controller	`HIGH`	Vanilla JS, Bootstrap	Giant file (2,001 lines), heavy DOM manipulation.
`work-match-skus/match-skus.mjs`	Spreadsheet SKU fuzzy matcher	`LOW`	Node.js, @oai/artifact-tool	Hardcoded developer local filesystem paths.

20. SECURITY ANALYSIS

Vulnerability Matrix

Vulnerability Category	Assessed Status	Evidence / Code Location	Risk Level
Authentication Bypass	`CONFIRMED ABSENT`	Zero auth checks on all 46 API routes in `app.py`.	`CRITICAL`
Public Exposure	`CONFIRMED OCCURRENCE`	`cloudflared.log` exposes port 5000 to public Internet.	`CRITICAL`
Remote Code Execution	`PROTECTED / LOW RISK`	Subprocess calls in `app.py:2627` use fixed command lists.	`LOW`
SQL Injection	`GENERALLY SAFE / MEDIUM`	Parameterized queries used; dynamic table names in ALTER TABLE.	`MEDIUM`
Server-Side Request Forgery (SSRF)	`WELL PROTECTED`	`validate_supplier_remote_url` validates domain + global IP.	`LOW`
Cross-Site Request Forgery (CSRF)	`PARTIALLY PROTECTED`	`reject_cross_origin_state_changes` blocks Origin mismatches.	`MEDIUM`
Path Traversal	`PROTECTED`	Safe path joining and allowlists on file download routes.	`LOW`

Vulnerability Category	Assessed Status	Evidence / Code Location	Risk Level
Secret Leaks	`CLEAN`	No hardcoded production passwords or API tokens discovered.	`LOW`

21. ERROR HANDLING & SILENT FAILURES

Forensic Findings `[VERIFIED]`

1. **Generic `except Exception:` Blocks:** Over 60 generic exception catches throughout `app.py`, `database.py`, and scraper engines.
2. **Missing Transaction Rollbacks:** When an error occurs during multi-row item insertion in `database.py:save\_fetch\_history`, no `conn.rollback()` is executed, leaving incomplete fetch records.
3. **Silent Automation Job Swallowing:** If a target URL returns a 404 or anti-bot block, errors are logged but the job marks the run as partially successful without alerting administrators.

22. LOGGING & OBSERVABILITY

- **Logging Framework:** Standard Python `logging` with `RotatingFileHandler` writing to `logs/server.log` (max 2MB per file, 5 backups).
- **Missing Observability:**
- No OpenTelemetry, Prometheus metrics, or Datadog tracing.
- No Sentry or exception monitoring integration.
- No structured JSON logging format.

23. PERFORMANCE ANALYSIS

1. **Memory Pressure from 50k+ Scrapes:** Querying all items from a large scrape loads 56,000+ dictionaries into Python heap memory simultaneously, consuming 350MB+ RAM per export.
 2. **Botasaurus Browser Overhead:** Each Chrome worker process spawned by Botasaurus consumes 250MB–500MB of RAM. Spawning 4 concurrent windows strains machines with <8GB RAM.
 3. **Un-Indexed URL Queries:** Queries filtering `items` on `site` or `created\_at` perform full table scans on tables with 50,000+ rows.
-

24. SCALABILITY ANALYSIS

Load Level	System Capability	Immediate Bottleneck / Failure Point
10 Users	`CAPABLE`	Responsive for read operations; slight delay on concurrent exports.
100 Users	`DEGRADED / FRAGILE`	SQLite write contention (`database is locked`); Gunicorn worker timeout.
1,000 Users	`FAILURE`	In-process scheduler thread crashes; memory exhaustion; browser starvation.
10,000 Users	`IMPOSSIBLE`	Single-node architecture cannot support multi-tenant distributed traffic.

25. CONCURRENCY & DATA INTEGRITY

Concurrency Vulnerabilities `[VERIFIED]`

1. **In-Process Scheduler Multi-Worker Hazard:** If Gunicorn runs with `--workers 2` or higher, each worker process starts its own `\_automation\_scheduler\_loop`, executing scheduled jobs multiple times simultaneously.
2. **SQLite Locking Contention:** When an automated background job writes 50,000 items in a loop, concurrent incoming web requests to `/api/watchlist` fail with `sqlite3.OperationalError: database is locked`.

26. CONFIGURATION ANALYSIS

- **Environment Settings** (`.env` vs `.env.example`):
- `.env` contains 20 tuning parameters` (`SCRAPER_USE_BROWSER=true`, `SCRAPER_LOCAL_BROWSER_MAX_WINDOWS=1`, `SCRAPER_ANOMALY_GUARD=true`).
- `.env.example` is present and documents most settings.`
- **Environment Coupling:** Hardcoded Windows paths in `work-match-skus/match-skus.mjs` make cross-platform execution impossible without manual edits.`

27. DEPLOYMENT ARCHITECTURE

Deployment Overview

1. **Local Desktop Deployment** (`start.bat`): Batch script for Windows 10/11 bootstrapping .venv`, scanning ports 5000–5050, and opening default browser.`
2. **Container Deployment** (`Dockerfile`): Docker image based on python:3.12.10-slim` with Chromium and Gunicorn.`
3. **Hosting Target:** Configured for Fly.io / single-node container runtime with `$PORT` binding on 8080.`

28. DEPENDENCY AUDIT

Dependency Inventory & Issues

Package	Declared Version	Installed Version	Status	Concern
`Flask`	`>=3.0.0,<4`	3.1.0	`HEALTHY`	Web framework.
`botasaurus`	`>=4.0.0`	4.0.97	`HEALTHY`	Anti-bot browser framework.
`openpyxl`	`>=3.1.0`	3.1.5	`HEALTHY`	Spreadsheet engine.
`curl_cffi`	`>=0.6.0`	**NOT INSTALLED**	`MISSING`	`fast_enrich_catalog.py` crashes on import.
`pandas`	`>=2.2.0`	**NOT INSTALLED**	`MISSING`	`run_menu_map_scrapers.py` crashes on import.
`Pillow (PIL)`	**NOT IN REQ.TXT**	**NOT INSTALLED**	`MISSING`	`compress_image_scraper_export.py` crashes.

29. CODE QUALITY ANALYSIS

Code Smells & Architectural Anti-Patterns

- **God Files (Extreme Monoliths):****
 - `app.py`: 4,613 lines (Mixes routing, scraping, threading, caching, XML generation, and business rules).
 - `database.py`: 3,402 lines (Mixes schema creation, migrations, CRUD, aggregations, and multi-DB routing).
 - `static/js/automation.js`: 2,669 lines (Monolithic JavaScript file managing complex UI state).
- **Tight Coupling:**** Route handlers in `app.py` directly instantiate scraper classes and access database global instances without dependency injection.

30. DUPLICATE LOGIC

- **Duplicate Price Parsing (`parse\_price\_number`):**** Implemented in `app.py`, `database.py`, `scraper\_engine.py`, `xcell\_scraper\_engine.py`, `txparts\_scraper\_engine.py`, `parts4cells\_scraper\_engine.py`, and `phonelcdparts\_scraper\_engine.py`.
- **Duplicate Text Cleaning (`clean\_text` / `strip\_markup`):**** Redefined across 6 distinct scraper engine files with slight regex variations.
- **Duplicate HTTP Header Definitions:**** Safari/Chrome user-agent strings duplicated across 8 files.

31. DEAD / LEGACY CODE

- **Dead Database Tables `[VERIFIED]`:****

- Tables `ms\_brands`, `ms\_categories`, `ms\_models`, `ms\_products`, `ms\_price\_history`, and `scraper\_runs` exist in all 8 database files with **0** rows.
 - These represent an abandoned attempt to build a normalized catalog hierarchy.
2. **Root** `mobilesentrrix.db` **VERIFIED**: Empty database file left in project root after migration to `data/site\_dbs/`.

32. TESTING ASSESSMENT

Pytest Suite Audit: 100 Tests Total (94 Passed, 6 Failed)

```
===== short test summary info =====
FAILED tests/test_api_scrape.py::test_scrape_always_uses_botasaurus_mode
FAILED tests/test_api_scrape.py::test_xcell_listing_does_not_auto_enable_slow_detail_scan
FAILED tests/test_api_scrape.py::test_running_automation_run_exposes_live_product_preview
FAILED tests/test_extractor_ui_regression_codex.py::test_automation_distinguishes_schedules_from_run_snapshots
FAILED tests/test_history_accessibility_contract.py::test_loading_overlay_uses_utility_class_and_live_status
FAILED tests/test_session_comparison.py::test_comparison_does_not_confirm_removed_from_single_missing_scrape
===== 6 failed, 94 passed in 82.21s (0:01:22) =====
```

Breakdown of the 6 Failing Tests:

1. **test_scrape_always_uses_botasaurus_mode**: `app.py` forces `use_browser=False` based on environment flags, failing an assertion expecting browser mode by default.
2. **test_xcell_listing_does_not_auto_enable_slow_detail_scan**: `execute_scrape_workflow` forces `enrich_details=True` for XCell unconditionally, violating the fast-scrape contract.
3. **test_running_automation_run_exposes_live_product_preview**: `/api/automation/runs/<run_id>` returns `payload["current_history"] = None` during live preview, throwing `TypeError: 'NoneType' object is not subscriptable`.
4. **test_automation_distinguishes_schedules_from_run_snapshots**: UI HTML in `templates/automation.html` was refactored and no longer contains the exact string `"Run History"`.
5. **test_loading_overlay_uses_utility_class_and_live_status**: `templates/history.html` is missing `<div id="overlay">`, causing `StopIteration` during DOM accessibility assertions.
6. **test_comparison_does_not_confirm_removed_from_single_missing_scrape**: `build_session_comparison` incorrectly flags an item as permanently removed (`removed=1`) on a single missing scrape instead of marking it suspect (`removed=0`).

33. MISSING FUNCTIONALITY

Definitely Missing (Required for Production)

- User Authentication & Role Management (Login, JWT, Session tokens)
- Centralized Distributed Queue (Redis + Celery / RQ)

- Multi-Worker Distributed Scheduler (Celery Beat / APScheduler with DB lock)
- Automated Database Backup / S3 Cloud Sync Routine
- API Key Authentication for External Integrations

Possibly Intended / Needs Confirmation

- Population of normalized catalog tables (`ms\_products`, `ms\_price\_history`)
- Webhook notifications on price drops (Slack, Discord, Email)
- Direct Shopify / WooCommerce inventory synchronization

34. FAILURE SCENARIO ANALYSIS (10 ADVERSARIAL CASES)

- **Scenario 1: Supplier Implements Cloudflare Turnstile / Captcha****
 - ***Expected Behavior:** Browser handles challenge.
 - ***Actual Failure:** If challenge exceeds 30s, worker raises `RuntimeError("Botasaurus remained on verification page")`, aborting scrape.
 - ***Severity:** `HIGH`
- **Scenario 2: Unicorn Deployed with 4 Workers****
 - ***Expected Behavior:** Web concurrency scales.
 - ***Actual Failure:** 4 separate scheduler threads fire, creating 4 duplicate scraping jobs simultaneously, triggering rate limits.
 - ***Severity:** `CRITICAL`
- **Scenario 3: SQLite Database Locked During Concurrent Write****
 - ***Expected Behavior:** Request waits for lock.
 - ***Actual Failure:** After 60s timeout, raises `sqlite3.OperationalError: database is locked`, returning 500 error to user.
 - ***Severity:** `HIGH`
- **Scenario 4: Server Process Killed During Automation Run****
 - ***Expected Behavior:** Run marks interrupted or resumes on restart.
 - ***Actual Failure:** Job remains in `running` state in DB until `recover\_running\_automation\_runs` executes on next startup.
 - ***Severity:** `MEDIUM`
- **Scenario 5: Malformed JSON Returned in Product Schema****
 - ***Expected Behavior:** Engine skips malformed script.

- ***Actual Failure:** Safely caught by JSON parser in `scraper\_engine.py:262`.
- ***Severity:** `LOW`

6. ****Scenario 6: Disk Space Exhaustion from Browser Profiles****

- ***Expected Behavior:** Profiles cleaned up.
- ***Actual Failure:** `data/browser\_profiles/` accumulates hundreds of megabytes of Chrome cache data indefinitely.
- ***Severity:** `MEDIUM`

7. ****Scenario 7: Supplier Changes Product Card HTML Classes****

- ***Expected Behavior:** Fallback selectors activate.
- ***Actual Failure:** Category scrape returns 0 items; Anomaly Guard triggers and rejects baseline.
- ***Severity:** `HIGH`

8. ****Scenario 8: User Submits Loopback / Private IP to Image Proxy****

- ***Expected Behavior:** Request blocked.
- ***Actual Failure:** `validate\_supplier\_remote\_url` verifies global IP and blocks request with 400 error.
- ***Severity:** `LOW (Successfully Mitigated)`

9. ****Scenario 9: Rapid Consecutive Deletes on Scrape History****

- ***Expected Behavior:** Records deleted safely.
- ***Actual Failure:** Absence of explicit transaction lock risks foreign key race conditions if concurrent writes occur.
- ***Severity:** `MEDIUM`

1. ****Scenario 10: Multi-User Web Traffic Exposes Unauthenticated APIs****

- ***Expected Behavior:** Auth blocks request.
 - ***Actual Failure:** Unauthenticated user deletes all history records via `/api/history/<id>/delete`.
 - ***Severity:** `CRITICAL`
-

35. ROOT CAUSE ANALYSIS

ID	ROOT CAUSE	EVIDENCE	PROBLEMS CREATED
RC-01	Monolithic Architecture without Layering	app.py (4.6k lines), db (3.4k)	Extreme refactor impossible test i
RC-02	In-Process Thread Scheduler Execution Model	_automation_scheduler_loop in app.py, single-worker Docker	Duplicate runs und multi-worker WSGI
RC-03	Total Omission of Authentication Layer	46 unauthenticated routes, trycloudflare public tunnel	Unauthorized data public abuse
RC-04	Dual Schema Architecture (Flat vs Normalized)	6 empty tables in 8 DBs, 56k rows in flat items table	Massive data redu performance degrad
RC-05	Missing Transactional Atomicity in Database	Raw cursor.execute without explicit rollback handlers	Orphan records, p crawl state corrup
RC-06	Unmanaged Dependencies in Helper Scripts	Missing curl_cffi, pandas, Pillow in requirements.txt	Broken CLI pipelin immediate script c
RC-07	Hardcoded Concurrency and Environment Limits	SCRAPER_LOCAL_BROWSER_MAX=1, Windows paths in mjs scripts	Severe scaling ces environment lock-

36. MAJOR ARCHITECTURAL AND TECHNICAL FLAWS

[CRITICAL-01] Complete Absence of Authentication on Destructive API Endpoints

- Severity:** `CRITICAL`
- Location:** `app.py:2980-4460` (All 46 routes)
- Description:** No authentication or authorization checks exist on any endpoint, including destructive endpoints like `/api/history/<id>/delete`, `/api/watchlist/clear`, `/api/cleanup`, and `/api/automation/jobs`.
- Evidence:** Inspection of route decorators confirms `@app.get` and `@app.post` without authentication middleware.
- Root Cause:** `RC-03`
- Recommended Direction:** Implement Flask-Login / JWT Bearer Token middleware and protect all state-changing endpoints.

[CRITICAL-02] Public Internet Exposure via Cloudflare Tunnel

- Severity:** `CRITICAL`
- Location:** `cloudflared.log`
- Description:** Cloudflare tunnel was launched against `http://localhost:5000`, publicly publishing the unauthenticated instance.
- Evidence:** `cloudflared.log` records active tunnel: `https://\*.trycloudflare.com`.
- Root Cause:** `RC-03`

- **Recommended Direction:** Immediately terminate unauthenticated quick tunnels and enforce Cloudflare Access Zero Trust with email/SSO authentication.

[CRITICAL-03] In-Process Scheduler Concurrency Hazard

- **Severity:** CRITICAL
- **Location:** app.py:2328-2665` (\_launch\_automation\_job`, `\_automation\_scheduler\_loop`)
- **Description:** Background jobs are scheduled and executed on in-memory threads. Deploying multi-worker Unicorn spawns multiple schedulers, causing duplicate scraping runs and supplier IP bans.
- **Evidence:** Dockerfile` line 23 explicitly notes: "Keep one worker so the in-process automation scheduler cannot duplicate jobs."
- **Root Cause:** RC-02
- **Recommended Direction:** Replace in-process thread loop with Celery + Redis or RQ with distributed task locks.

[HIGH-01] Six Automated Test Suite Regressions

- **Severity:** HIGH
- **Location:** tests/test_api_scrape.py`, tests/test\_session\_comparison.py`, tests/test_extractor_ui_regression_codex.py`, tests/test\_history\_accessibility\_contract.py`
- **Description:** 6 test cases fail due to forced scraping parameters, DOM element ID mismatches, NoneType errors on live preview, and session comparison bugs.
- **Evidence:** Pytest output: 6 failed, 94 passed.
- **Root Cause:** RC-01
- **Recommended Direction:** Fix `payload["current\_history"]` None check, align DOM template IDs, and correct session comparison remove counter.

[HIGH-02] Missing Package Dependencies in Utility Scripts

- **Severity:** HIGH
- **Location:** scripts/fast_enrich_catalog.py`, scripts/full\_pipeline\_fast.py`, scripts/run_menu_map_scrapers.py`, scripts/compress\_image\_scraper\_export.py`
- **Description:** Scripts import `curl\_cffi`, `pandas`, and `PIL` (Pillow) which are not installed in the environment or listed in `requirements.txt`.
- **Evidence:** Code AST imports verified against `pip list`.
- **Root Cause:** RC-06
- **Recommended Direction:** Add `Pillow`, `pandas`, and `curl\_cffi` to `requirements.txt`.

37. MAJOR MISSING COMPONENTS

- 1. **Authentication & Identity Provider Layer:** JWT / Session auth.
- 2. **Distributed Asynchronous Task Queue:** Celery / Redis worker architecture.
- 3. **Enterprise Relational Database:** PostgreSQL with connection pooling.
- 4. **Structured API Rate Limiting:** Flask-Limiter with Redis backend.
- 5. **Database Migration Pipeline:** Alembic / Flask-Migrate version control.
- 6. **APM & Centralized Logging:** Structured JSON logging + Sentry error tracking.
- 7. **CI/CD Automated Testing Pipeline:** GitHub Actions workflow.

38. TECHNICAL DEBT REGISTER

ID	Technical Debt Description	Location	Impact	Difficulty	Priority
TD-01	Refactor 4,613-line `app.py` into Flask Blueprints	`app.py`	High	High	P1
TD-02	Refactor 3,402-line `database.py` into Service/Repository layer	`database.py`	High	High	P1
TD-03	Fix 6 failing automated pytest regression tests	`tests/`	High	Low	P1
TD-04	Drop or populate 6 dead database tables (`ms_brands`, etc.)	`database.py`	Medium	Medium	P2
TD-05	Consolidate duplicate price parsing and text cleaning functions	`scrapers/*`	Medium	Low	P2
TD-06	Replace hardcoded Windows paths with dynamic configuration	`work-match-skus`	Medium	Low	P3
TD-07	Implement automated Chrome browser profile cache cleanup	`data/browser_profiles`	Medium	Low	P3

39. POSITIVE ARCHITECTURAL DECISIONS

- 1. **Domain-Isolated SQLite Persistence:** Storing each supplier's catalog in a separate SQLite database prevents cross-site data pollution and enables independent database backups.
- 2. **Strict SSRF Network Boundary Defense:** `validate\_supplier\_remote\_url` validates domain allowlists, resolves DNS, and verifies `ipaddress.ip\_address.is\_global`, effectively stopping intranet scanning.
- 3. **Automated Anomaly Detection Guard:** The Anomaly Guard algorithm stops accidental baseline destruction if a supplier blocks a scraper or serves an empty page.

4. ****Hybrid HTTP + Headless Browser Fallback:**** Fast Safari TLS requests run first for speed, falling back to Botasaurus Chromium only when anti-bot challenges are detected.
5. ****Rich Formatted Spreadsheet Generation:**** `openpyxl` exporter builds clean, branded Excel sheets with embedded formulas and direct image hyperlinks.

40. PRODUCTION READINESS ASSESSMENT

Classification: FUNCTIONAL MVP / EARLY STAGE (NOT PRODUCTION READY)

****Verdict:**** The system is an impressive, highly functional MVP for single-user desktop operations. However, it is ****unfit for multi-user production deployment**** due to:

1. Zero authentication on all endpoints.
2. An in-process thread scheduler that cannot scale to multi-worker WSGI deployments.
3. SQLite database write lock contention under concurrent user operations.
4. 6 failing automated regression test cases.
5. High refactoring risk due to monolithic 4,000+ line code files.

41. SYSTEM MATURITY SCORE

Category	Score (0–10)	Forensic Justification
Architecture	4 / 10	Monolithic god files; in-process scheduler; lacks distributed task queues.
Code Quality	5 / 10	Clean styling but massive files, duplicate utility functions, and tight coupling.
Database Design	4 / 10	Good WAL pragmas and multi-DB isolation, but flat denormalized schema and dead tables.
Security	3 / 10	Excellent SSRF defense, but 0 authentication and public tunnel exposure.
Reliability	6 / 10	Resilient Botasaurus fallback and Anomaly Guard, but SQLite locking hazards.
Scalability	3 / 10	Hardcoded 1–4 browser slots; single-node in-memory thread execution model.
Maintainability	4 / 10	Difficult to refactor without breaking existing workflows due to file size.
Testing	6 / 10	100 tests exist (94 passing), but 6 failing tests indicate active regressions.
Monitoring	3 / 10	Basic rotating file logging; lacks structured metrics, APM, and alerting.
Deployment	5 / 10	Functional Dockerfile and start.bat, but restricted to 1 WSGI worker.
Documentation	6 / 10	Multiple markdown architecture documents present in `docs/`.
OVERALL SCORE	**4.5 / 10**	**Functional Prototype / Early MVP requiring architectural hardening.**

42. PRIORITY MATRIX

```
+-----+
| 1. FIX IMMEDIATELY (Next 24-48 Hours) |
|   - Implement API Authentication (JWT / Session Token / API Keys) |
|   - Terminate unauthenticated Cloudflare quick tunnels |
|   - Fix 6 failing pytest regression tests |
|   - Fix missing dependencies in requirements.txt (Pillow, pandas, curl_cffi) |
+-----+
| 2. FIX BEFORE SCALING (Next 2-4 Weeks) |
|   - Migrate background jobs from in-process threads to Celery + Redis |
|   - Refactor app.py into Flask Blueprints (Scraper, Automation, History, Export) |
|   - Refactor database.py into Repository / Unit-of-Work pattern |
|   - Implement PostgreSQL for centralized metadata and catalog storage |
+-----+
| 3. IMPROVE SOON (Next 1-2 Months) |
|   - Consolidate duplicate price parsing and text cleaning utilities |
|   - Clean up dead database tables (ms_brands, ms_products, etc.) |
|   - Implement Sentry error monitoring and Prometheus metrics |
|   - Add GitHub Actions CI/CD test automation pipeline |
+-----+
| 4. OPTIONAL IMPROVEMENTS (Backlog) |
|   - Build webhook notifications for real-time price drop alerts |
|   - Add direct Shopify / WooCommerce catalog sync integrations |
|   - Containerize browser instances into dedicated remote browser grid |
+-----+
```

43. TOP 20 QUESTIONS FOR THE NEXT ARCHITECT

1. What was the intended role of the normalized `ms\_brands` -> `ms\_products` hierarchy, and should it be resurrected or purged?
2. Why is `xcell\_scraper\_engine` forcing detail enrichment on category listing crawls when other scrapers make it optional?
3. What is the target production user concurrency (single desktop user vs multi-tenant SaaS team)?
4. Should PostgreSQL replace the multi-file SQLite architecture for multi-user production?
5. Why are background jobs managed via in-process threads rather than a distributed Celery queue?
6. Is Cloudflare Access configured on the domain when tunnels are active in production?
7. What is the retention policy for historical scrapes, and why are 70+ database backup snapshots accumulating in `data/backups`?
8. How should the system handle dynamic Cloudflare Turnstile captchas that Botasaurus cannot solve within 30 seconds?
9. Why does `work-match-skus/match-skus.mjs` rely on private `@oai/artifact-tool` packages and hardcoded local Windows paths?
 1. Should the frontend migrate from server-rendered Jinja templates to a modern React / Vue SPA?
 1. How should price change alerts be broadcast (WebSockets vs SSE vs Webhooks)?

1. Why are 6 tests failing in the test suite, and were these regressions introduced during the v8 recovery?
 1. Can the 7 scraper engines share a unified base scraper class rather than duplicating parsing logic?
 1. What is the strategy for browser user profile lifecycle management to prevent disk bloat?
 1. Is there a business requirement to support proxy rotation (residential / datacenter proxies) for large supplier crawls?
 1. How should supplier-side schema/layout drifts be detected and alerted automatically?
 1. Why is `openpyxl` used for Excel generation instead of faster streaming writers for 50k+ row datasets?
 1. Should the Anomaly Guard threshold be user-configurable per supplier rather than global environment variables?
 1. How should rate-limiting be enforced against suppliers to avoid IP blacklisting?
 1. What CI/CD pipeline will be used to enforce automated testing prior to container deployments?
-

44. RECOMMENDED INVESTIGATION AREAS

1. ****Verify Botasaurus Anti-Bot Bypass Success Rate:**** Measure live success rates against MobileSentry US/CA and XCellParts Cloudflare challenges under continuous execution.
 2. ****Inspect Memory Leakage in Long-Running Background Runs:**** Monitor Python process RSS memory growth during multi-thousand product scrapes.
 3. ****Audit Database Backup Growth:**** Review automated backup routines in `data/backups/` to establish a formal retention and pruning schedule.
 4. ****Evaluate Celery / Redis Migration Effort:**** Estimate refactoring scope to decouple background threads into standalone worker containers.
-

45. SPECIAL SECTION: HANDOFF TO INDEPENDENT TECHNICAL REVIEWER

Critical Focus Areas for Secondary Review

- ****Examine `app.py` Lines 2328–2665:**** Audit the in-process scheduler implementation and verify why it cannot run safely under multi-worker Gunicorn.
- ****Inspect `database.py` Lines 680–740:**** Verify missing multi-table transaction rollback protection during batch item insertion.
- ****Review Pytest Failures in `tests/test\_api\_scrape.py`:**** Confirm whether the failure on live preview payload (`payload["current\_history"]`) is caused by asynchronous race conditions.
- ****Inspect `scrapers/browser\_fetcher.py` Lines 88–116:**** Audit the thread semaphore slot allocation logic to ensure no deadlocks occur when browser instances crash.

- ****Verify `validate\_supplier\_remote\_url` in `app.py:2848`:**** Confirm the strength of the SSRF filter against DNS rebinding attacks.

46. APPENDIX

Appendix A: Important File Index

- `app.py`: Main Flask application, 46 route controllers, scheduler loop (4,613 lines).
- `database.py`: DatabaseManager & MultiDatabaseManager SQLite facade (3,402 lines).
- `automation\_service.py`: Automated category discovery and URL crawling helpers (787 lines).
- `scrapers/scrapper\_engine.py`: MobileSentry core scraper and JSON-LD parser (1,126 lines).
- `scrapers/xcell\_scraper\_engine.py`: XCellParts specialized scraper engine (1,013 lines).
- `scrapers/browser\_fetcher.py`: Botasaurus browser slot allocator and Cloudflare handler (269 lines).
- `scrapers/registry.py`: Shared supplier domain mapping and configuration (120 lines).
- `scrapers/menu\_map/common.py`: Menu hierarchy crawler and auto-healing framework (1,146 lines).
- `Dockerfile`: Debian-slim container definition with Gunicorn and Chromium (26 lines).
- `start.bat`: Windows desktop automated launcher and environment validator (278 lines).

Appendix B: Complete API Catalog

(See full catalog of 46 endpoints in Section 14)

Appendix C: Database Table List

1. `automation\_jobs`: Configured recurring scraping jobs (Active in `mobilesentry.db`).
2. `automation\_job\_targets`: Discovered/assigned category target URLs per job (5,717 rows in `mobilesentry.db`).
3. `automation\_runs`: Execution run history, status, and summary metrics.
4. `fetch\_history`: Scrape session records with timestamp, URLs, and pricing rules.
5. `items`: Flat scraped product records (56,906 rows in `mobilesentry.db`, 11,646 in `xcellparts.db`).
6. `watchlist\_items`: User-saved bookmarked products for quick monitoring.
7. `ms\_brands`: Normalized brand catalog table (Unused, 0 rows).
8. `ms\_categories`: Normalized category catalog table (Unused, 0 rows).
9. `ms\_models`: Normalized device model catalog table (Unused, 0 rows).
1. `ms\_products`: Normalized master product catalog table (Unused, 0 rows).
1. `ms\_price\_history`: Normalized historical product pricing table (Unused, 0 rows).
1. `scraper\_runs`: Normalized scraper run record table (Unused, 0 rows).

Appendix D: Environment Variable Names (Values Redacted)

- `SCRAPER\_USE\_BROWSER`
- `SCRAPER\_LOCAL\_BROWSER\_ENGINE`
- `SCRAPER\_LOCAL\_BROWSER\_HEADLESS`
- `SCRAPER\_LOCAL\_BROWSER\_MAX\_WINDOWS`
- `SCRAPER\_LOCAL\_BROWSER\_WAIT\_SECONDS`
- `SCRAPER\_BROWSER\_ONLY`
- `SCRAPER\_BOTASAUROS\_REQUEST\_HTML`
- `SCRAPER\_XCELL\_MAX\_WORKERS`
- `XCELL\_MAX\_WORKERS`
- `SCRAPER\_ANOMALY\_GUARD`
- `SCRAPER\_BASELINE\_PROTECTION`
- `SCRAPER\_MAX\_TOTAL\_DROP\_RATIO`
- `SCRAPER\_MAX\_TOTAL\_DROP\_ITEMS`
- `SCRAPER\_MAX\_TARGET\_ERROR\_RATIO`
- `SCRAPER\_HISTORY\_KEEP\_PER\_URL\_SET`
- `SCRAPER\_ANOMALY\_MIN\_PREVIOUS`
- `SCRAPER\_ANOMALY\_SPARSE\_RATIO`
- `SCRAPER\_ANOMALY\_MAX\_SPARSE\_ITEMS`
- `SCRAPER\_CHATGPT\_AUTO\_REPORT`
- `SCRAPER\_CHATGPT\_MODEL`
- `DATABASES\_DIR`
- `PUBLIC\_BASE\_URL`
- `CORS\_ALLOWED\_ORIGINS`
- `PORT`
- `WEB\_WORKERS`
- `WEB\_THREADS`
- `WEB\_TIMEOUT`

Appendix E: External Service Inventory

- **Target Suppliers:** MobileSentry US, MobileSentry CA, XCellParts, TXParts, Parts4Cells, PhoneLCDParts, GadgetFix.
- **Local Services:** Headless Chromium instances managed via Botasaurus.
- **Tunnels:** Cloudflare Tunnel (`cloudflared`).

Appendix F: Background Job Inventory

1. **Automation Scheduler Loop** (`_automation_scheduler_loop`): Checks job schedules every 15s.
2. **Category Scraper Worker** (`_launch_automation_job` worker`): Executes two-phase category & detail extraction.
3. **Menu Map Crawler Worker** (`run_menu_map_scrapers`): Executes headless category hierarchy discovery.
4. **Proxied Image Cache Cleaner** (`cleanup_proxied_image_cache`): Evicts expired image buffers from memory.

Appendix G: Important Business Rules

- Pricing markup & discount calculation formula with rounding to 2 decimal places.
- Anomaly Guard baseline rejection if product drop exceeds 50%.
- Metadata hydration from previous crawl sessions to avoid redundant detail page fetches.
- SSRF global IP verification blocking all RFC 1918 private subnets.

Appendix H: Code Quality & Marker Analysis

- Developer `TODO` / `FIXME`` comments: 0 found in codebase.
- Dead tables: 6 empty tables across 8 database files.
- Large files: `app.py`` (4,613 lines), `database.py`` (3,402 lines), `automation.js`` (2,669 lines).

Appendix I: Potentially Unused Files

- `mobilesentry.db`` (Root legacy database: 0 rows).
- `work-match-skus/match-skus.mjs`` (Standalone utility with hardcoded paths).
- `work-match-skus/cleanup-old-outputs.mjs`` (Standalone utility).

Appendix J: High-Risk Files

- `app.py``: Contains entire web API, scheduler, and business logic without auth.
- `database.py``: Manages all database connections without transactional rollbacks.
- `cloudflared.log``: Exposes public tunnel access history.
- `scrapers/browser_fetcher.py``: Manages browser automation processes and semaphore locks.

End of Forensic Technical Audit Report.